

Rapport de test JUnit /S.A.E 2.01

méthode testées : Récolter()

```
public boolean recolter(Mine mine) { 1 usage  ⚙ Abderrahmane BAHLOUL
    if (mine == null) {
        return false;
    }

    if (mine.getTypeMinerai() != typeMinerai) {
        return false;
    }

    if (mine.estVide() || estPlein()) {
        return false;
    }

    int placeDisponible = capaciteStockage - stockActuel;
    int quantiteDemandee = Math.min(capaciteExtraction, placeDisponible);
    int quantiteExtraite = mine.extraire(quantiteDemandee);

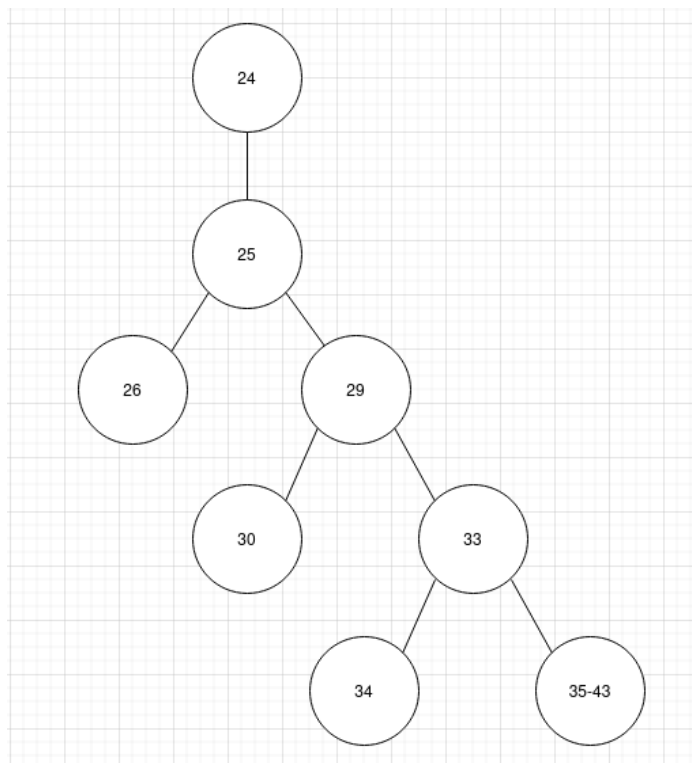
    stockActuel += quantiteExtraite;

    return quantiteExtraite > 0;
}
```

Objectif des tests :

La méthode recolter() sert à extraire une mine compatible avec le robot. On vérifie que la méthode fonctionne dans les cas importants : récolte réussie ; mine inexistante ; mauvais type de minerai ; mine vide ou entrepôt plein.

Graphe de flot :



Jeu de Test:

Lien	Données	Résultat attendu
(24,25),(25,26) il n'y a pas de mine.	mine == null	false
(24,25),(25,29),(29,30) Il y a une mine mais elle n'est pas du même type de minerai que le robot.	robot or + mine nickel	false
(24,25),(25,29),(29,33),(33,34) Il y a une mine, la mine est compatible avec le robot mais l'entrepôt est plein ou la mine est vide.	mine.stockActuel = 0	false
(24,25),(25,29),(29,33),(33,35-43) Toutes les conditions sont remplies.	mine != null, robot or + mine or, mine.stockActuel = 50, capaciteStockage = 10, robot.stockActuel = 6, capaciteExtraction = 3	True

Code Junit :

```
package sae.model;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class RobotRecolterTest {
    @Test
    void recolter_echoue_siMineNull() {
        Robot robot = new Robot(1, TypeMinerai.OR, 0, 0, 10, 3);

        boolean resultat = robot.recolter(null);

        assertFalse(resultat);
    }
}
```

```

        assertEquals(0, robot.getStockActuel());
    }

    @Test
    void recolter_echoue_siMauvaisTypeMine() {
        Robot robot = new Robot(1, TypeMinerai.OR, 0, 0, 10, 3);
        Mine mineNickel = new Mine(1, TypeMinerai.NICKEL, 0, 0,
50);

        boolean resultat = robot.recolter(mineNickel);

        assertFalse(resultat);
        assertEquals(0, robot.getStockActuel());
        assertEquals(50, mineNickel.getStockActuel());
    }

    @Test
    void recolter_echoue_siRobotPlein() {
        Robot robot = new Robot(1, TypeMinerai.OR, 0, 0, 10, 3);
        Mine mineOr = new Mine(1, TypeMinerai.OR, 0, 0, 50);

        robot.recolter(mineOr);
        robot.recolter(mineOr);
        robot.recolter(mineOr);
        robot.recolter(mineOr);

        boolean resultat = robot.recolter(mineOr);

        assertFalse(resultat);
        assertEquals(10, robot.getStockActuel());
        assertEquals(40, mineOr.getStockActuel());
    }

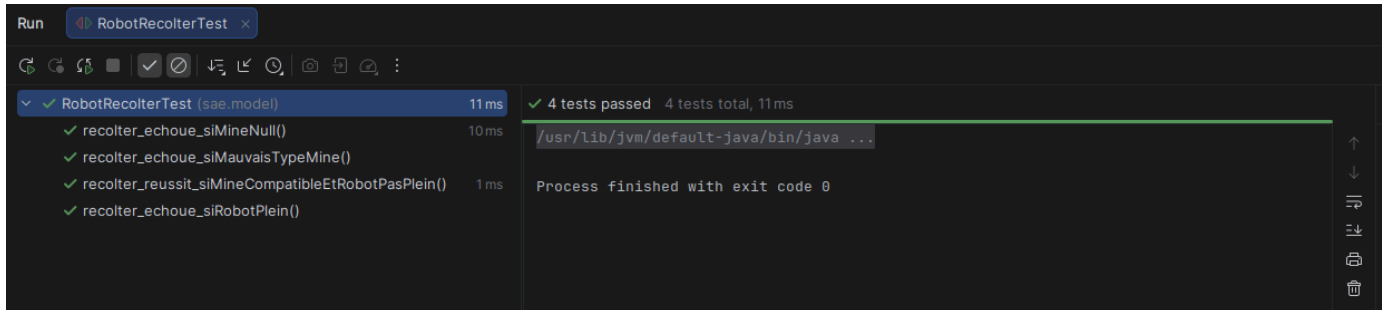
    @Test
    void recolter_reussit_siMineCompatibleEtRobotPasPlein() {
        Robot robot = new Robot(1, TypeMinerai.OR, 0, 0, 10, 3);
        Mine mineOr = new Mine(1, TypeMinerai.OR, 0, 0, 50);

        robot.recolter(mineOr);
        boolean resultat = robot.recolter(mineOr);

        assertTrue(resultat);
        assertEquals(6, robot.getStockActuel());
        assertEquals(44, mineOr.getStockActuel());
    }
}

```

Résultat des tests :



Analyse des résultats :

Les tests montrent que la méthode recolter() fonctionne correctement dans les cas normaux et que les erreurs sont bien gérées.

Un robot peut récolter :

- dans une mine compatible avec son type de minerai,
- lorsque son stock n'est pas plein et qu'il reste du minerai dans la mine.

Un robot ne peut pas récolter :

- si la mine est inexistante (mine == null),
- dans une mine d'un type de minerai différent,
- lorsque son stock est déjà plein.

Méthode testées: déposer()

Objectif des tests :

L'objectif est de vérifier que la méthode déposer () fonctionne correctement dans tous les cas importants : dépôt réussi ; entrepôt inexistant ; mauvais type d'entrepôt ; robot vide.

Code de la méthode testées:

```
public boolean deposer(Entrepot entrepot) { 5 usages
    if (entrepot == null) {
        return false;
    }

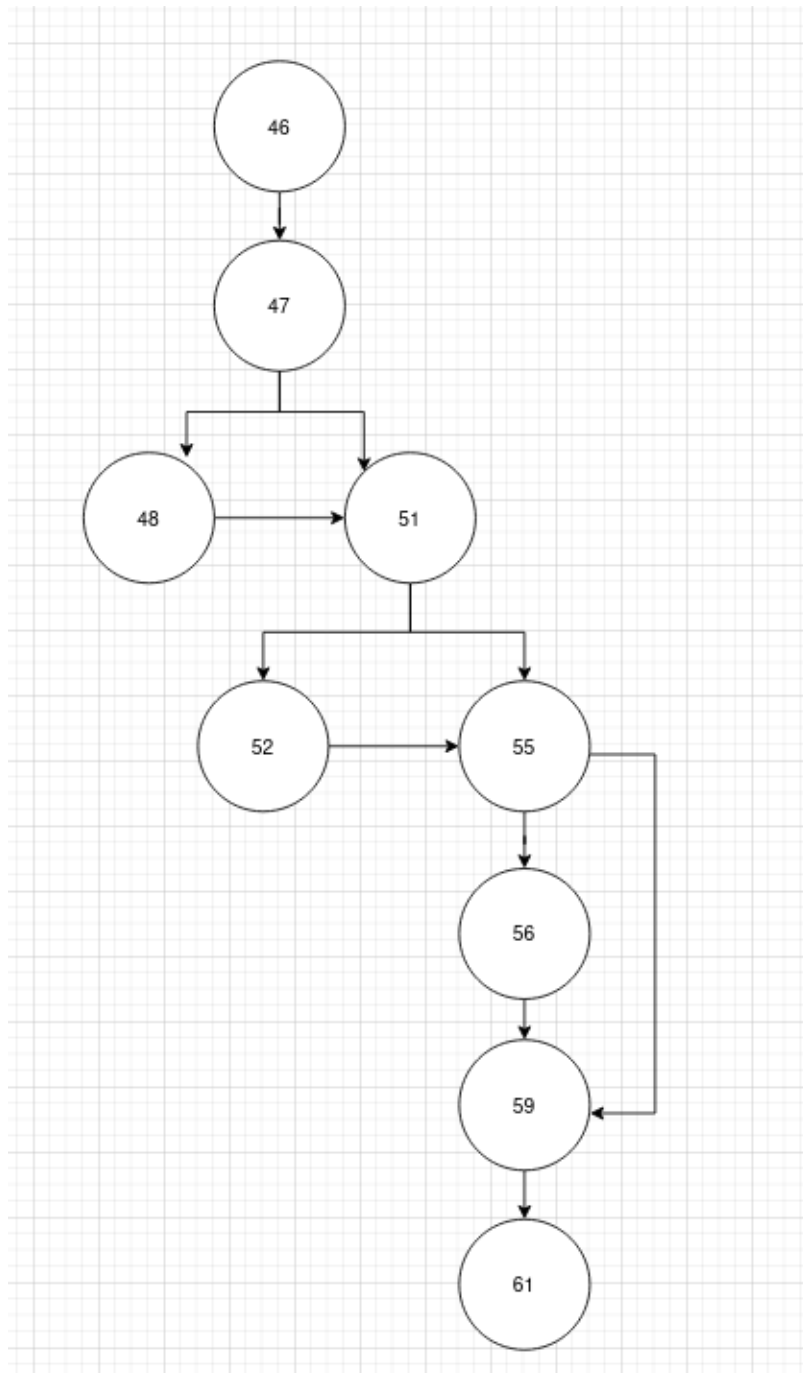
    if (entrepot.getTypeMinerai() != typeMinerai) {
        return false;
    }

    if (estVide()) {
        return false;
    }

    entrepot.ajouterMinerais(stockActuel);
    stockActuel = 0;

    return true;
}
```

Graphe de flot :



Code Junit :

```
package sae.model;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class RobotDeposerTest {

    @Test
    void deposer_reussit_siEntrepotMemeTypeEtRobotAvecStock() {
        Robot robot = new Robot( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0, capaciteStockage: 5, capaciteExtraction: 2);
        Mine mine = new Mine( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0, stockInitial: 10);
        Entrepot entrepot = new Entrepot( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0);

        robot.recolter(mine);

        boolean resultat = robot.deposer(entrepot);

        assertTrue(resultat);
        assertEquals( expected: 0, robot.getStockActuel());
        assertEquals( expected: 2, entrepot.getStock());
    }

    @Test
    void deposer_echoue_siEntrepotNull() {
        Robot robot = new Robot( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0, capaciteStockage: 5, capaciteExtraction: 2);

        boolean resultat = robot.deposer( entrepot: null);

        assertFalse(resultat);
        assertEquals( expected: 0, robot.getStockActuel());
    }
}
```

```
@Test
void deposer_echoue_siMauvaisTypeEntrepot() {
    Robot robot = new Robot( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0, capaciteStockage: 5, capaciteExtraction: 2);
    Mine mine = new Mine( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0, stockInitial: 10);
    Entrepot entrepotNickel = new Entrepot( numero: 1, TypeMinerai.NICKEL, ligne: 0, colonne: 0);

    robot.recolter(mine);

    boolean resultat = robot.deposer(entrepotNickel);

    assertFalse(resultat);
    assertEquals( expected: 2, robot.getStockActuel());
    assertEquals( expected: 0, entrepotNickel.getStock());
}

@Test
void deposer_echoue_siRobotVide() {
    Robot robot = new Robot( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0, capaciteStockage: 5, capaciteExtraction: 2);
    Entrepot entrepot = new Entrepot( numero: 1, TypeMinerai.OR, ligne: 0, colonne: 0);

    boolean resultat = robot.deposer(entrepot);

    assertFalse(resultat);
    assertEquals( expected: 0, robot.getStockActuel());
    assertEquals( expected: 0, entrepot.getStock());
}
```

Jeu de Test:

Situation testées	Données	résultat attendu	résultat obtenu
Dépôt normal	Robot or avec stock + entrepôt avec stock	true	true
Entrepôt null	null	false	false
Mauvais type	robot or + entrepôt nickel	false	false
robot vide	stock =0	false	false

Résultat des tests :

```
✓ RobotDeposerTest (sae.model) 10ms
  ✓ deposer_echoue_siEntrepotNull() 9ms
  ✓ deposer_reussit_siEntrepotMemeType() 1ms
  ✓ deposer_echoue_siRobotVide()
  ✓ deposer_echoue_siMauvaisTypeEntrepot()

✓ 4 tests passed 4 tests total, 10ms
/usr/lib/jvm/default-java/bin/java ...

Process finished with exit code 0
```

Analyse des résultats:

Les tests montrent que :

- la méthode fonctionne correctement dans le cas normal ;
- les erreurs sont correctement gérées ;

Un robot ne peut pas déposer :

- dans un entrepôt incompatible ;
- sans stock ;
- dans un entrepôt inexistant.